NodeJS Lockfile Adoption: Hardening against supply chain attacks

```
#begin-approvals-addon-section

See go/g3a-approvals for instructions on how to add reviewers. Do not edit this
section manually.
```

Author: Gabriel Pearhill | Last Updated: Jun 2, 2026 | Status: Draft
Project: <link to go/cloud-sdk-idea> | Self link: go/sdk:<name>

Objective

Secure the `google-cloud-node` monorepo against supply chain attacks and optimize our CI with unified dependency management.

Background

Where we are today

The `google-cloud-node` repository is a multi-package monorepo containing over 260 client library packages for Google Cloud APIs.

Currently, the repository is configured to ignore package lockfiles. Lockfiles pin dependency versions (with hashes to verify integrity), preventing packages from automatically being upgraded when new minor versions release.

- `package-lock.json` and `*.lock` are gitignored in the root `.gitignore` and in individual package directories (e.g., `core/common/.gitignore`).
- CI/CD pipelines go subdirectory-by-subdirectory, resolving and installing dependencies (defined in `ci/run_single_test.sh`) without a pre-existing lockfile often automatically installing the latest minor version of each package.

This strategy was adopted to reduce the maintenance burden of managing dependencies (and to catch failures early). However, this also leaves us vulnerable to supply chain attacks like Shai Hulud (explained below).

The threat

A dynamic resolution model leaves the monorepo highly exposed to supply chain attacks. To understand what this threat looks like, consider a recent example: Shai Hulud. This cyber attack (and follow ups using a similar strategy) work like this:

1. **Account Takeover**: Attackers hijack the npm credentials of legitimate maintainers of widely used transitive packages.
2. **Malicious Script Payloads:** The attacker publishes compromised versions containing invasive lifecycle scripts (e.g., `preinstall`, `install`, or `postinstall` in `package.json`).
3. **Credential Harvesting:** When a developer or a CI runner executes a dynamic `npm install`, the lifecycle script automatically fires a payload to scrape local environments for secret keys:
    - npm publish tokens
    - GitHub Personal Access Tokens (PATs)
    - Cloud provider credentials (GCP service account keys, AWS keys)
    - SSH credentials
4. **Self-Propagation:** Using stolen npm credentials, the worm can infect and republish other packages managed by the hijacked developer.

Without a lockfile, our CI/CD runs an unprotected NPM install on every build. The moment a compromised transitive package is published within our semver ranges (e.g., `^1.0.0`), our next build will pull it, execute the malware, and potentially compromise credentials.

Overview

Mitigation (using a lockfile)

By committing a vetted lockfile to Git, we establish three layers of cryptographic defense:

- **Strict Version Pinning:** Every dependency—both direct and transitive—is locked to an exact, vetted version. Fresh releases on the registry are ignored until a developer explicitly commits an upgrade.
- **Cryptographic Integrity:** Lockfiles record cryptographic hashes for all files. If a package is modified or replaced on the registry at an existing version (registry poisoning), the installer halts immediately on hash mismatch.
- **Scope and Registry Pinning:** Binds dependencies to specific registries, preventing dependency confusion attacks where attackers attempt to register public versions of internal private package names.

High level plan

Adopt **PNPM Workspaces** with a single, unified `pnpm-lock.yaml` at the root of the repository. This will involve the following steps:

1. **PNPM Workspaces:** We will define a single workspace structure. pnpm natively dedupes packages and provides excellent performance. A single root lockfile avoids the massive reviews and overhead associated with maintaining 260+ independent lockfiles.
2. **Configuration Updates:** We will modify .gitignorerules to commit the lockfile and pnpm configuration.
3. **CI Pipeline Refactoring:** We will shift from multiple, sequential installs inside each package folder to a single, root-level workspace install (`pnpm install --frozen-lockfile`) executed once at the start of the workflow.
4. **Renovate Bot Integration:** Renovate will be updated via renovate.json to monitor and update the root lockfile in weekly, grouped intervals to prevent pull request fatigue.

Detailed Design

Establishing the Workspace Configuration

We will declare a single unified workspace by creating a `pnpm-workspace.yaml` at the root of the repository:

```
None
packages:
  - 'core'
  - 'core/packages/*'
  - 'core/dev-packages/*'
  - 'packages/*'
  - 'handwritten/*'
  - 'containers/*'
```

The following install will generate a global `pnpm-lock.yaml` file for us.

Update .gitignore to track global lockfile, ignore others

This requires the following steps:

1. Remove per-package .gitignore of lock files (this will be done globally now).
2. Add a global ignore for all lockfiles other than the root `pnpm-lock.yaml`. To protect against accidental inclusion, we will ignore all common lockfile formats: PNPM (outside of root lockfile), NPM, Yarn, and Bun.

Pipeline and Script Modifications

Root CI Execution: Update `.github/workflows/presubmit.yaml` to execute a single, cached frozen installation step right after setup:

```
None
      - uses: pnpm/action-setup@v4
        with:
          version: ^10.0.0
      - name: Install Workspace Dependencies (Frozen Lockfile)
        run: pnpm install --frozen-lockfile
```

Refactoring Single Tests: Modify `ci/run_single_test.sh` to remove local installs:

```
None
# Dependencies are pre-installed globally at the workspace root.
# We only execute compilation / prep if required by the package.
pnpm run compile --if-present
```

Long-Term Lockfile Maintenance

Automated Updates (Renovate Bot): Configure renovate.json to handle `pnpm-lock.yaml` updates:

- Pin transitive updates.
- Group updates into a single weekly PR to avoid PR noise.

Enforcing Lockfile Synchronicity: Enforce the `--frozen-lockfile` flag in CI. Builds will fail if a contributor edits a dependency in package.json but forgets to update the root pnpm-lock.yaml locally.

Security Scans: Leverage our existing security scan infrastructure to identify vulnerabilities and issue PRs for targeted upgrades (this is done with Dependabot today). TODO: See if Renovate Bot can simply perform this action as well.

Impact on Consumers and Local Developers

Consumers (Our Customers): Zero impact. Lockfiles are strictly for repository development. They are automatically omitted from published npm registry tarballs. Consumers installing our libraries (e.g. `npm install @google-cloud/alloydb`) will continue to use their own package managers (NPM, Yarn, PNPM, Bun) and resolve packages accordingly.

Local Monorepo Contributors:

- Contributors should run `pnpm install` at the repository root to bootstrap their local environment. This sets up crucial local workspace symlinks (e.g. linking `packages/google-cloud-alloydb` to a local edit in `core/common`).
- Running npm install inside a subdirectory will fail to identify the workspace structure, downloading published code from the registry instead of linking local packages.
- Once bootstrapped at the root, contributors are fully supported to navigate to subdirectories and run local npm/pnpm scripts (npm test, npm run compile) normally.

They can also execute commands from the root: `pnpm --filter @google-cloud/alloydb` test.

Alternatives considered

Distributed Per-Package Lockfiles (packages/*/pnpm-lock.yaml)

Instead of a root lockfile, we considered committing individual lockfiles into each of the 260+ subdirectories.

Why not this option?

- **High Maintenance Burden**: Renovate or Dependabot would have to open, review, and merge dozens of PRs every week to update lockfiles across hundreds of folders, causing overwhelming developer fatigue.
- **Slow CI:** CI pipelines would have to execute hundreds of isolated sequential pnpm install steps, dragging pipeline duration up.
- **No Global Deduplication:** Prevents sharing identical transitive versions across different client packages, wasting storage and memory.

Adopting GitHub Dependabot instead of Renovate

We considered using GitHub's native Dependabot to automate lockfile upgrades.

Why not this option?

- **PR Volume:** Dependabot lacks the advanced grouping rules of Renovate. It would open a separate PR for every single minor/patch bump, clogging the PR queue and testing pipelines.
- **Monorepo Awareness:** Dependabot's workspace optimization is far less robust than Renovate's.